R.real.10

Complément Web

JS + DOM

M4103C : organisation

3 CM : 1h20

- Intro + Manipulation de la DOM
- React
- React Native

3 TD :

- 3h DOM
- 3h React Base
- 3h React SPA (Single Page Application)
- 1h20 React Native (Non évélué)

1 DS :

- 2h

JavaScript : le langage

ECMA-262

- Langage orienté objet
- Interprété
- Faiblement typé
- Exécuté principalement coté client

- https://developer.mozilla.org/fr/docs/Web/JavaScript

JavaScript : comment charger du JS

- Dans une balise

```
<element eventHandler="script" />
```

- Dans l ’entête de la page

```
<script type="text/javascript" src= "truc.js"></script>
```

- Dans la page

```
<script type="text/javascript">
…
</script>
```

JavaScript : éléments du langage

- Commentaires
  - // ligne
  - /* bloc */
- Constantes
  - NaN, Infinite, Undefined
- Variable
  - Portée : le bloc
  - var nom [=valeur]
  - Types : booléens, indéfini, nombre, null, chaîne de caractère, symboles, objects (et Arrays)

JavaScript : éléments du langage

- Déclaration des variables
  - let a = 1;
  - a = function() {}
- Déclaration des constantes
  - const a = 10;

Rem : Langage par référence, « use strict »;

JavaScript : éléments du langage

- Quels sont les types de :

```
let a = 2;
console.log(typeof a);
let b= a +"4";
console.log(typeof b);
let c=  parseFloat(b)
console.log(typeof c);
let d = ["Un", "Deux", "Trois"];
console.log(typeof d);
let e = {nom:"Lestideau", prenom:"vincent"};
console.log(typeof e);
```

JavaScript : éléments du langage

Des objets globaux

- **Number**, BigInt,Math,Date
- **String**, RegExp
- **Array**, Int8Array, Uint8Array, …
- **Map, Set**, WeakMap, WeakSet
- ArrayBuffer,SharedArrayBuffer ,Atomics ,DataView, **JSON**
- **Promise**, Generator, GeneratorFunction ,**AsyncFunction**

…

JavaScript : éléments du langage

- Tableaux indicés
  - Indicés à partir de 0
  - var tab = [elt1, elt2, …] ou var tab = **new** Array(elt1, elt2, …) //Les tableaux sont des objets
  - accès par l'indice tab[indice]

- Tableaux associatifs
  - tab= new Array(); tab[clef1]=elt1, tab[clef2]=elt2
  - objets littéraux :
    - var tab = {

      clef1: elt1,

      clef2: elt2,

      };
    - accès par le nom de l'attibut via un point(.) tab.clef1
- Mélange des 2 type possible
- tab.length (longeur), tab.sort() (tri), unshift(elt) (ajout au début), push(elt) (ajout à la fin), shift() (retire en début), pop() retire en fin, …

JavaScript : éléments du langage

```
let toto = {
        photos: {        page: 1,
                         pages: "1017",
                         perpage: 4,
                         total: "4065",
                         photo : [
                                  { id: "33276227384",
                                  owner: "95362858@N03",
                                  secret: "eea1701366",'
                                  server: "2928",
                                  farm: 3,
                                  title: "Nice Art + Paddy_4739 avenue du Général Jean Simon Paris 13",
                                  ispublic": 1,
                                  isfriend: 0,
                                  isfamily: 0 }
                                  ]},
        "stat": "ok" }';
```

- Comment accéder stat de data ?
- Comment accéder au owner de la première case du tableau

JavaScript : éléments du langage

Fonctions

- Appel nomFonction(paramètres)

- Declaration

```
function nomFonction(paramètres){
code
}
```

Ou let fonction = function() {}

- Fonction anonyme

```
function (paramètres){
code
}
```

- Return pour renvoyer une valeur

- Auto exécution avec () à la fin pour la forme var f = function(){}();

- Attention une déclaration de variable sans var crée une variable globale qui existe après l'exécution de la fonction

JavaScript : éléments du langage

Une fonction est une variable

```
function maFonction(arg1, arg2)
{ // code
  return valeur;
}
```

ou

```
var maVar = function(arg1, arg2)
{
      // code
                return valeur;
}
```

Avantage redéfinition possible en modifiant la variable, tableaux de fonctions, …

JavaScript : éléments du langage

- Valeur par défaut des arguments

```
function multiplier(a, b = 1) { return a * b; }
multiplier(5, 2); // 10
multiplier(5, 1); // 5
multiplier(5); // 5
```

- Nombre variable d'arguments :
  - test avec undefined
  - Nombre illimité d'arguments : maFonction.arguments est un tableau indicé contenant les arguments

JavaScript : éléments du langage

Que donne :

```
const test1= function() {
        let x = 1;
        test2 = function() {
                let x = 2;
                test3 = function() {
                        let x = 3;
                        console.log(x);
                }();
        }();
        console.log(x);
}();
console.log(x); //Affiche dans la console
```

JavaScript POO minimaliste

- JS est un langage par prototype, nous verrons cela dans un autre module.
  - Une fonction est un constructeur auto déclarant

```
let Animal = function(nom){
    this.nom = nom || "toto";
}
Animal.prototype.crie = function () {
    return this.nom+" crie";
}
const jojo = new Animal("Jojo");
console.log(jojo, jojo.nom, jojo.crie());
```

JavaScript POO minimaliste

Possibilité également de définir des classes avec variables privées, variables et méthodes statiques, héritage, …

```
class Animal2 {
   #nom;
   constructor(nom) {
      this.#nom = nom || toto;
   }
   crie() {
      return this.#nom+" crie";
   }

   get nom() {
      return this.#nom.toUpperCase();
   }

   toString() {
      return "M. "+this.#nom;
   }
}

let doudou = new Animal2("doudou");
console.log(doudou.nom);
console.log(doudou); //asseceur
console.log(doudou+"");
console.log(doudou.crie);
console.log(doudou.crie());
```

JavaScript POO minimaliste

Les fonctions des fonctions sans this qui peuvent accéder au this englobant.

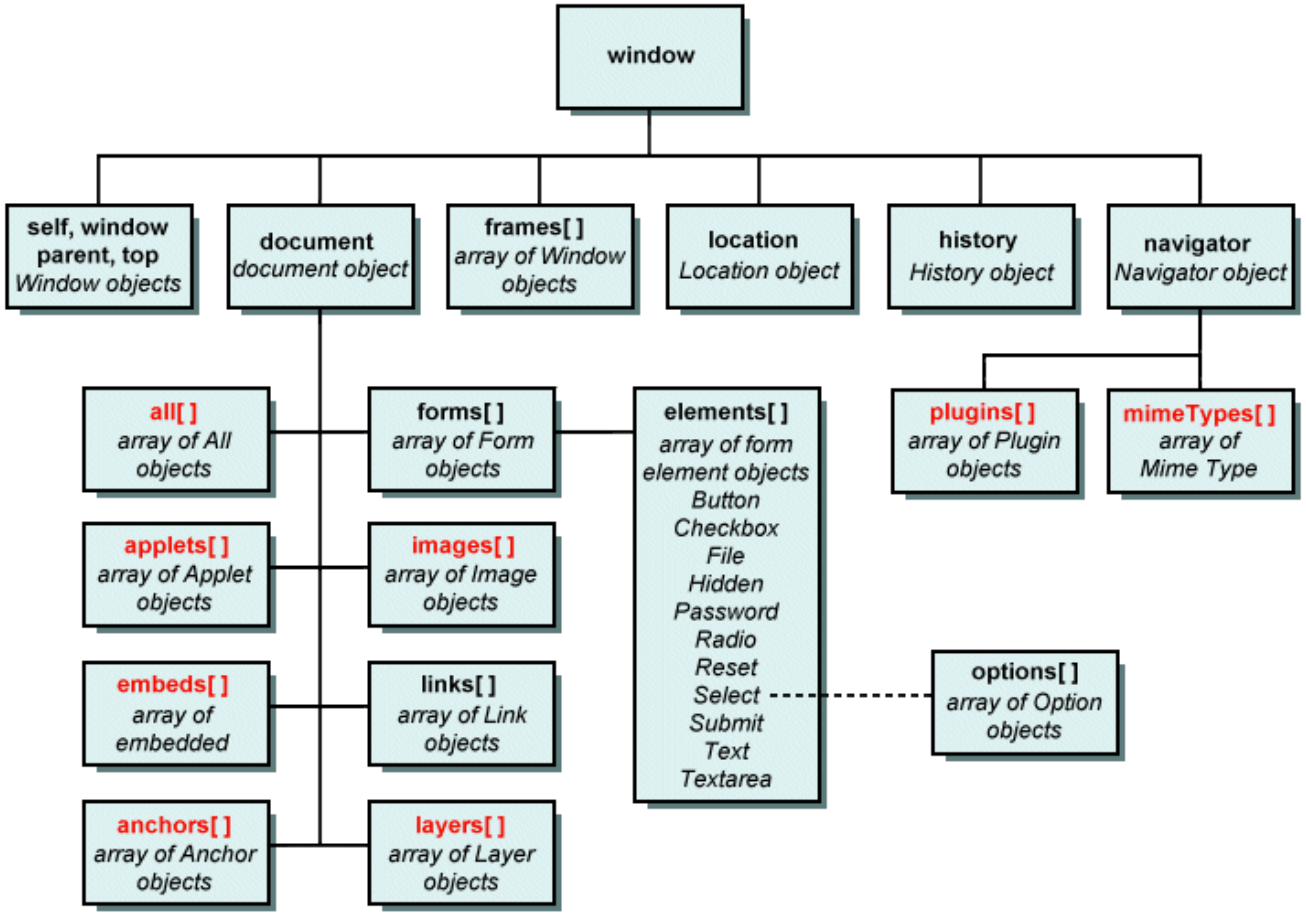
```
function Personne () {
        this.age = 0;
        setInterval(function grandir () {
                this.age++; //pb
   }, 1000); }
```

JavaScript POO minimaliste

Les fonctions des fonctions sans this qui peuvent accéder au this englobant.

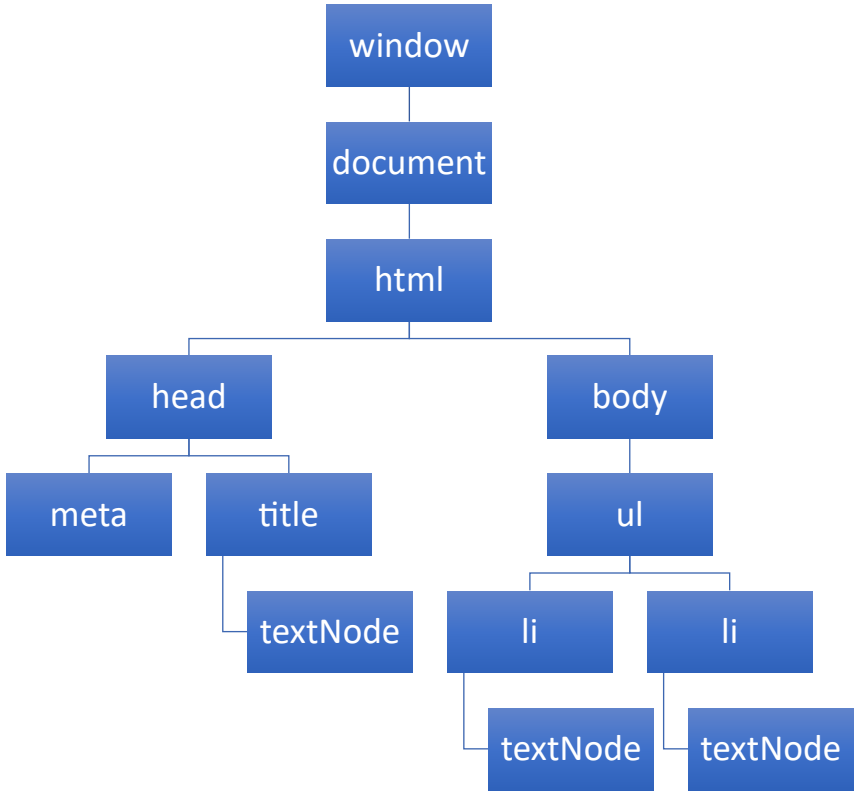
```
function Personne () { this.age = 0;
    setInterval(() => {
        this.age++;
    }, 1000); }
```

JavaScript : DOM



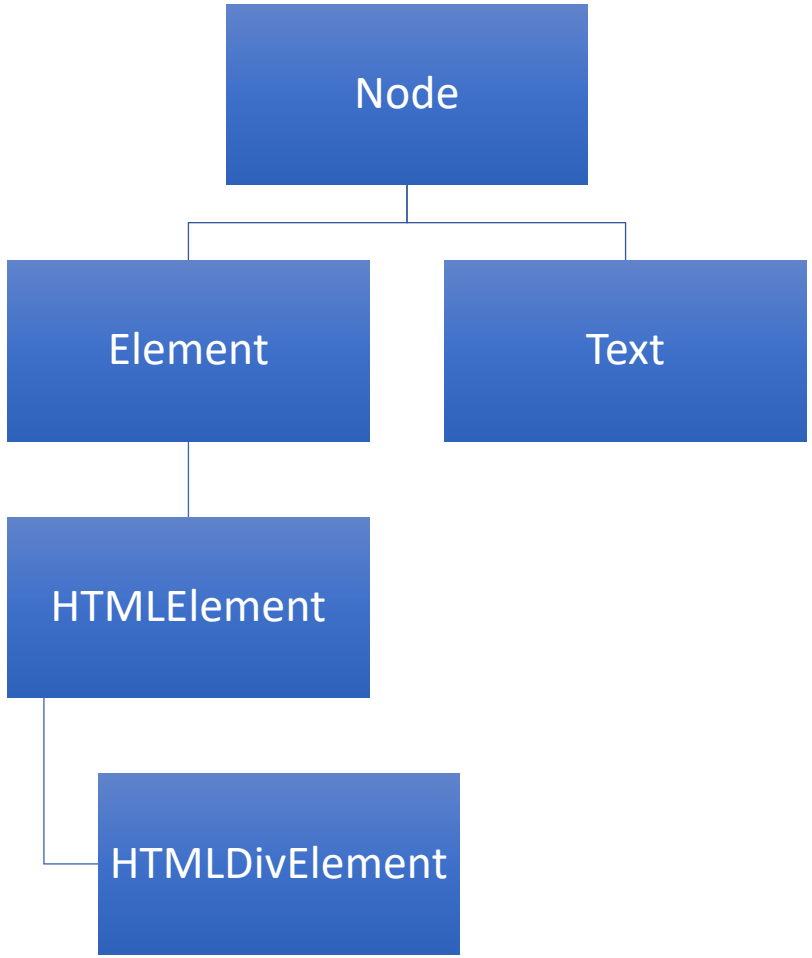
JavaScript : DOM

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Dom 1</title>
</head>
<body>
 <ul>
            <li> item 1 </li>
            <li> item 2 </li>
 </ul>
</body>
</html>
```



JavaScript : DOM

- 2 principaux tyes :
  - ElementNode
  - TextNode
- Notion d’héritage



JavaScript : DOM

Numéro (nodeType)	Type de nœud
1	Nœud élément
2	Nœud attribut
3	Nœud texte
4	Nœud pour passage CDATA (relatif au XML)
5	Nœud pour référence d'entité
6	Nœud pour entité
7	Nœud pour instruction de traitement
8	Nœud pour commentaire
9	Nœud document
10	Nœud type de document
11	Nœud de fragment de document
12	Nœud pour notation

JavaScript : DOM manipulation

- Accéder aux éléments
  - getElementById()
  - getElementsByTagName()
  - getElementsByName()

  - **querySelector()**
  - **querySelectorAll()**
- Accéder et modifier les attributs
  - getAttribute()
  - setAttribute()
  - className car class est un mot clef réservé
- Accéder, modifier le contenu
  - innerHTML
  - innerText

JavaScript : DOM manipulation

- Naviguer dans les nœuds
  - parentNode
  - nodeType
  - nodeName
  - firstChild,
  - lastChild
  - nodeValue ou data
  - childNodes
  - nextSibling
  - previousSibling
  - hasChild()

JavaScript : DOM manipulation

- Création
  - document.createElement()
  - document.createTextNode()
  - appendChild()
  - cloneNode()
  - insertBefore()
- Suppression remplacement
  - removeChild()
  - replaceChild()

JavaScript : programmation événementielle

Définition : en informatique, la programmation événementielle est un paradigme de programmation fondé sur les événements. Elle s'oppose à la programmation séquentielle. Le programme sera principalement défini par ses réactions aux différents événements qui peuvent se produire.

JavaScript : programmation événementielle

Un objet Event :

- Mouse Events
  - onclick
  - oncontextmenu
  - ondblclick
  - onmousedown
  - onmouseenter
  - onmouseleave
  - onmousemove
  - onmouseover
  - onmouseout
  - onmouseup
- Keyboard Events
- Frame/Object Events
- Form Events
- Drag Events
- …
- La méthode preventDefault() permet de bloquer les événements

JavaScript : programmation événementielle

Comment utiliser les événements :

1. Cliquez-moi ! <!– A EVITER -->
2. <script> let element = document.getElementById('clickme'); **element.onclick** = function() { console.log("Vous m'avez cliqué !"); }; </script> <!– A EVITER -->
3. <script> let element = document.querySelector(‘#clickme'); **element.addEventListener**(**'click**', function() {console.log("Vous m'avez cliqué !");}); </script>

JavaScript : programmation événementielle

- object.event = function() {} //fonction anonyme
- console.log(message) //pour afficher dans la console

- Faire afficher dans la console un message lorsque la fenêtre a finie de se chargée et lorsqu'elle est redimensionnée

JavaScript : Demo 1

- Afficher dans la console une information lorsque la fenêtre est chargée ou lorsqu'elle est redimensionnée

JavaScript : Mise en pratique

- En partant d’une page avec un body vide, créer un formulaire avec select et un input de type texte qui permet d’ajouter un item dans une liste à la positon choisie par le select

